

Hani Abusharkh (habusharkh)_PreviewUser Proposal.docx

Document Details

Submission ID

trn:oid::2945:414476677

Submission Date

Sep 22, 2026, 6:51 AM GMT+0

Download Date

Sep 22, 2026, 6:53 AM GMT+0

File Name

Proposal.docx

File Size

38.0 KB

2 Pages

429 Words

2,416 Characters

Research Proposal

Reconstructed Jira histories for late sprint closure and unfinished committed scope

Background

Sprint Planning matters because it's one of the last points where a team can still change course. Once execution starts, scope is mostly locked in. Repository data could help flag problems early — but only if it's handled carefully. The features have to reflect what the team actually knew at planning time, not details that only surfaced afterward.

Research aim

This study has two goals. First: find out whether Jira history can meaningfully rank two outcomes — sprints that close late, and committed scope that goes unfinished. Second, and simpler: check whether starting scope on its own already explains most of whatever signal is actually there.

Research questions

1. How much signal is really there for late closure and unfinished committed scope?
2. Does a simple scope-only model hold up against more flexible ones?
3. Are the results stable over time, and across different projects?
4. How much do gaps in historical provenance, or leakage-prone evaluation choices, change the findings?

Data

TAWOS is the dataset — a relational collection of public Jira projects. Eight of those projects make up the working subset here. Three tables do the heavy lifting: Sprint, Issue, and Change_Log. Together, they let key fields get rebuilt back to the exact moment each sprint started.

Method

Sprint membership and Story Points won't be taken at face value from a final export — they'll be rebuilt from timestamped change history instead. Sprints that never closed get dropped first, before any labels exist. Four approaches go head to head: logistic regression, random forest, XGBoost, plus simple prevalence and scope-only baselines. For evaluation, the plan is forward-in-time splits alongside held-out-project transfer, to see whether the results actually generalize.

Evaluation

ROC-AUC is the primary measure here. F1, precision, recall, PR-AUC, and Brier score come along too, each with its limitations flagged rather than hidden. To really pressure-test the conclusions, the study also digs into project-level uncertainty, multiple temporal cut points, sensitivity to provenance quality, feature permutation importance, and a handful of deliberate leakage demonstrations.

Expected contribution

The contribution here is as much about method as it is about prediction. It's meant to work as a reproducible case study — one that shows how timing, scope provenance, class prevalence, project differences, and leakage each quietly shape what sprint-analytics results actually mean.

Limits

This study is retrospective, and it leans entirely on open-source repositories. It won't claim that a risk score actually improves Scrum decisions — that's a bigger claim, and one that would need real teams and prospective validation to back it up.